TASK-SCOPED PHYSICAL UNDO: HISTORY-CONDITIONED RESTORATION WITH PROTECTION CONSTRAINTS

Author information pending

Author affiliations pending
Working draft — 20 October 2026

ABSTRACT

Cancelling a completed household task requires deciding which effects still belong to it after later activities. We formulate Task-Scoped Physical Undo: infer restoration and protection constraints from history, then satisfy them through ordinary robot actions. Matched histories can share the same current state and cancellation request while requiring different goals. We separate exact goal inference, predicted-goal completion, and reference physical task success. On six English development conditions from three simulated physical histories, model-extracted facts followed by the specified rule achieve six joint successes for both tested models; direct goals achieve zero for Qwen and five for Phi-4. Goal-level agent adapters reveal additional program and representation errors. Separately, a geometry-based generator completes ten of sixteen development histories across scene, asset and history-combination conditions. The results support history-sensitive evaluation while exposing physical generation and independent generalization as remaining challenges.

***Index Terms*—** Embodied agents, selective undo, task cancellation, historical attribution, robot manipulation

1. INTRODUCTION

A robot prepares a table for dinner. Later, a household member reserves one cup for personal use and places unrelated belongings on the table. Cancelling dinner should return the unused dinnerware while preserving the adopted cup and the later contributions. Clearing the table, restoring an earlier snapshot, or reversing all original actions can violate these requirements. The robot must first determine which effects still belong to the cancelled task.

We formulate *Task-Scoped Physical Undo*: a user identifies a completed task, and an agent infers and physically satisfies its remaining restoration obligations while protecting valid later activity. The goal is defined by historical responsibility rather than a fixed tidy state. Two histories can have the same current scene and cancellation request but require different actions. For example, reserving either a book or a cup without moving it changes which object should be returned

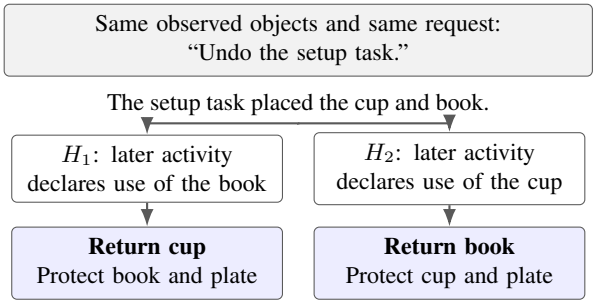


Fig. 1. A schematic history contrast. The target task placed both objects; only the later use declaration changes. Both branches require a physical return and protect the unrelated plate. This illustrates the task, not a new simulation trial.

(Fig. 1). Such pairs test historical attribution while holding current geometry fixed.

Our contributions are an explicit placement-domain specification; matched-history evaluation connected to actual simulated actions; and separate measurement of goal inference, execution and history generation. We compare direct model goals, model-extracted facts followed by a specified rule, and goal-level agent/program adapters. The rule is a specification baseline, not a claimed new reasoning algorithm.

On six English development conditions, facts + rule achieves six joint successes for each tested model, while direct prediction is sensitive to model and output representation. Separately, the current history generator completes ten of sixteen development candidates across scene, asset and history-combination conditions. This provides cross-condition evidence; a fresh independent generalization test remains outstanding.

2. RELATED WORK

Selective undo is established in collaborative software: Shao et al. [1] account for other operations when undoing a selected one. Harland et al. [2] formalize goal/plan cancellation and termination behavior. In robotics, Has et al. [3] plan kitchen cleanup after abort commands. Rewind-IL [4] recovers from execution failure to resume the original task. We instead study

cancelling a completed task after later activities have adopted some of its effects.

BEHAVIOR-1K [5] and PARTNR [6] provide rich household execution and collaboration settings. Our implementation uses OmniGibson and BEHAVIOR-1K assets. ReAct [7] and Code as Policies (CaP) [8] motivate the history-query and generated-program adapters; these are goal-level adaptations, not complete reproductions. PDDLStream [9], MoveIt Task Constructor [10], and affordance-based task and motion planning [11] motivate separating geometric candidates and connected manipulation stages. We use cuRobo [12] as the motion backend; this engineering basis does not establish a new planning algorithm.

3. TASK AND EVALUATION

3.1. Restoration and protection semantics

An episode provides (H, s_u, q_τ): a complete ordered history, the undo-start observation, and a request cancelling completed task τ. Object and support identities are fixed. Tasks are sequential, later placements and explicit use declarations persist, and no external activity occurs during undo. The domain covers reversible placements; expired claims, disputed or temporary moves, nested cancellation, blocked restoration destinations, cabinet states and irreversible processes are excluded. Declarations are authored evidence, not inferred human intentions.

For registered object o, let $b_\tau(o)$ be its support immediately before the *whole* task starts, $W_\tau(o)$ its successful placements by τ, and $L_\tau(o)$ placements or use declarations by other tasks after its last placement by τ. The contract is

$$R_\tau = \{o : W_\tau(o) \neq \varnothing,\ L_\tau(o) = \varnothing\}, \tag{1}$$

$$P_\tau = \mathcal{O} \setminus R_\tau, \tag{2}$$

$$g_\tau = (\{o \mapsto b_\tau(o) : o \in R_\tau\}, P_\tau). \tag{3}$$

Unrelated objects are protected. Repeated placements retain the boundary before the entire task; earlier independent activity can make that boundary differ from the episode's initial state. The target task's own claims do not survive cancellation. The empirical domain excludes target effects ending on their own pre-task support. These are normative benchmark rules, not universal assumptions about human ownership.

A matched pair has identical (s_u, q_τ) but distinct exact contracts. A deterministic state-only predictor returns the same contract for both, so it can be exactly correct on at most one. This motivates history contrasts without treating paired branches as independent samples.

3.2. Physical criteria and distinct success measures

A checkpoint may initialize an undo trial. Afterwards, only normal navigation, grasping, placement, release and retreat may change the world; rollback and direct pose assignment are prohibited. Restoration completion C is the fraction of required objects released on their reference supports. Protection violation V is the fraction of protected objects grasped, moved to another support, translated over 2 cm or rotated over 5° from their undo-start poses. Supports and poses are checked at recorded control steps, with grasp checks at recorded physics substeps. This is sampled monitoring over registered objects, not a continuous-time guarantee for the entire room. Empty-set conventions are $C = 1, V = 0$; a legitimate no-op still requires a recorded wait and normal termination.

We distinguish exact goal correctness G, audited completion of the *predicted* contract E, and reference physical success:

$$G = \mathbf{1}[\hat{g} = g_\tau], \tag{4}$$

$$T = E \wedge (C = 1) \wedge (V = 0) \wedge \text{empty hands}, \tag{5}$$

$$J = G \wedge T. \tag{6}$$

A wrong goal can satisfy E. Every valid primary prediction is executed regardless of G. Invalid or missing predictions remain in the planned denominator. Abnormal or unscorable executions fail T, J; unavailable C, V remain missing rather than becoming zero violations.

4. SYSTEM AND METHODS

4.1. History, goal and execution interfaces

The history builder executes a recipe and validates actual events before admitting a history. English narratives are bound to successful event identifiers and preserve checkpoint bytes. Authored no-motion declarations create contrast branches. The goal module receives the public narrative, object/support catalogs, current supports, request and rules, but no private reference labels. A shared executor realizes the submitted contract; an offline evaluator checks the trace (Fig. 2).

A separately implemented forward obligation ledger supplies labels. Agreement with the deployed rule on 68,040 bounded symbolic combinations is a software consistency check, not additional physical episodes or independent human assessment.

4.2. Goal-construction methods

Structured rule computes Eq. (3) from exact events and the target ID. It is privileged relative to the language methods. **Direct** predicts an object-wise goal table and performs one fixed review without correctness feedback. **Facts + rule** first asks the model for ordered events with evidence references, then for the cancelled task and a goal. The rule readout uses those extracted facts and the model-selected task. It therefore still depends on model interpretation. The second call

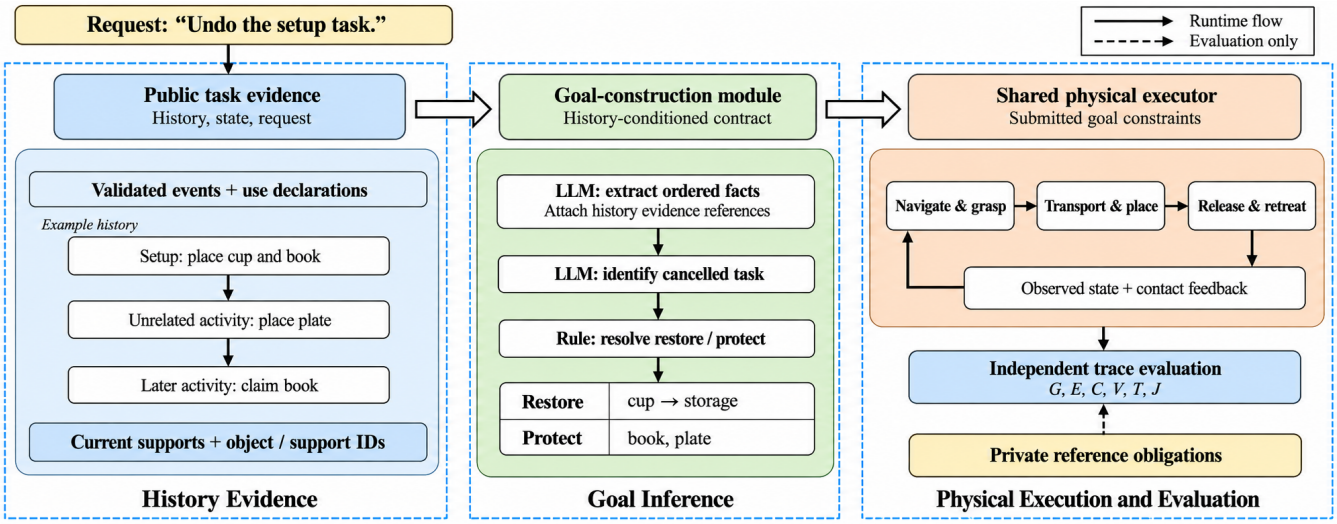


Fig. 2. System architecture with the facts + rule readout expanded. English evidence is converted into a restoration/protection contract and executed through ordinary robot actions. Direct and agent readouts replace the middle module while sharing the executor. Dashed arrows carry private reference obligations only to the evaluator. The cup/book contract is an illustrative example.

also supplies a *shared model readout*, enabling comparison on identical extracted evidence without extra calls.

The primary models are Qwen3-VL-8B-Instruct [13] and Phi-4 [14], both given text-only inputs. Their parameter counts differ. Each arm uses two calls, temperature zero and at most 3,072 output tokens per call. Controls compare table versus mapping outputs and a no-history direct readout. Matching extraction prompts are accompanied by an explicit check of saved extraction equality.

A separate Phi-4 study adds **ReAct** history queries and **CaP** program generation. Queries return public evidence, not attribution answers. CaP generates a Python function that returns the goal, without a supplied history parser or rule solver. Both permit at most four calls; direct and facts + rule retain two. Actual call counts are reported. These zero-shot goal adapters omit the original methods' physical-action interfaces. The CaP adapter executes generated programs in isolated CPython 3.10 with the standard library. It supports normal Python language semantics; the program receives only public inputs and cannot access private labels, the repository, network services or robot-control APIs. Program and output-validation errors can be returned to the model within its call budget.

4.3. Physical execution and history generation

All primary methods share Tiago, OmniGibson, sticky grasping and cuRobo. The executor enforces its *submitted* protection contract; gold labels are reserved for scoring, so wrong goals can produce observable mistakes. Undo allows one attempt, seed zero, 12,000 control steps and 2,400 seconds.

History generation requires normal successful grasps and placements for every prescribed move. Event validation checks observed postconditions, order, task boundaries, action counts and the final checkpoint. The current geometry policy searches support packing and workspace orientation from object dimensions and spatial constraints, without scene-name branches. Candidate layouts must satisfy support boundaries and object separation before initialization preview. The selected manipulation controller then executes the prescribed activity sequence. Asset identity, scale, activity order and outer budgets are retained for every candidate.

5. EXPERIMENTAL DESIGN

The main cohort contains three exposed physical ancestors and two no-motion adoption branches per ancestor: six English conditions. They cover a preparation of two objects followed by an independent placement, two independent preparations, and cancellation of a middle task after an earlier valid placement. One condition needs two restorations; another legitimately needs none. Methods and paired branches share checkpoints, and observed initial states are checked for equality. Translations and branches do not increase the ancestor count.

The main evaluation reports thirty executed method–condition cells: five methods on six conditions, with one execution per cell. It includes all semantic errors and physical failures of the selected methods. The two model services are run separately with the same histories, prompt structure and execution budgets. Run-level provenance is retained with the saved requests, responses and traces.

Table 1. English end-to-end evaluation: six conditions per method, with a shared executor and budget. G: exact goal; E: predicted-goal completion; T: reference physical success; J: joint success.

Method	G	E	T	J
Structured rule	6/6	6/6	6/6	6/6
Qwen direct	1/6	4/6	1/6	0/6
Qwen facts + rule	6/6	6/6	6/6	6/6
Phi-4 direct	6/6	5/6	5/6	5/6
Phi-4 facts + rule	6/6	6/6	6/6	6/6

Five language arms across six conditions and two models use 120 calls; shared readouts reuse them. The goal-only comparison evaluates the selected versions of four methods on the same six conditions, giving twenty-four method–condition cells. These goal evaluations do not supply additional physical results or independent sources.

The current *development-time transfer* cohort contains sixteen candidates from fifteen ancestral recipes, arranged as a $2 \times 2 \times 2$ scene/asset/history-combination matrix in two blocks. Scenes vary architectural shells but reuse inserted task furniture; assets are new instances within known categories. Longer histories combine prior activity, repeated target placements and later independent placement. These candidates have participated in debugging, so this is cross-condition development evidence, not a fresh independent test.

Every candidate has one 1,000-step/600-second preview and one 24,000-step/5,400-second complete-history attempt. All failures remain in the sixteen-candidate denominator. History yield, undo conditional on an admitted history, and success of history plus both undo branches are distinct quantities. Paired undo on the reconstructed cohort remains incomplete; its generation score cannot be combined with the main cohort's undo score.

Protocols freeze inputs, versions, budgets and seeds. Raw calls, traces, checkpoints, group mappings and independent scores are retained. We report descriptive counts, without IID confidence intervals over correlated branches. Unfinished diagnostic queues are outside the result tables.

6. RESULTS

6.1. Historical judgment and physical outcomes

The structured rule and both facts–rule methods achieve $G = E = T = J = 6/6$. Qwen direct produces one exact goal and completes four predicted contracts, yet has no joint successes. Its only exact goal fails physically, while a different, incorrect goal happens to produce a reference-compatible physical outcome; hence $G = T = 1/6$ but $J = 0/6$. Of five valid wrong goals, four complete their predicted contract and three of those fail the reference task. Successful execution of a submitted goal is therefore insufficient evidence of correct undo. Phi-4 direct produces six exact goals and five joint successes.

Among scorable executions, Qwen direct has mean $C = 0.750, V = 0.333$ ($n = 4$). The other methods have $C = 1, V = 0$ on their scored executions ($n = 6$ each, except Phi-4 direct: $n = 5$). Unscorable failures remain in T, J, not in these conditional means. Fig. 3 shows how changed adoption evidence produces different successful undo actions.

Table 2. Exact goals under the language controls. Each readout uses six conditions; shared model and rule readouts reuse the same calls and extracted evidence.

Readout	Qwen	Phi-4
Table direct	1/6	6/6
Table shared model	1/6	5/6
Table shared rule	6/6	6/6
Mapping direct	1/6	1/6
Mapping shared model	0/6	0/6
Mapping shared rule	6/6	6/6
No history	1/6	1/6

Table 3. Phi-4 goal inference on six conditions. Valid counts schema-valid goals; groups require both branches correct. Call counts include all attempts. ReAct and CaP are goal-level adapters.

Method	G	Valid	Groups	Calls
Direct	6/6	6/6	3/3	12
Facts + rule	6/6	6/6	3/3	12
ReAct goal adapter	5/6	5/6	2/3	12
CaP goal adapter	0/6	3/6	0/3	22

6.2. Extraction and representation controls

Both models extract exact facts and select the correct task in all six shared table conditions. Yet the shared model goals are correct in only one Qwen condition and five Phi-4 conditions, versus six for both rule readouts. These errors occur downstream of the recorded extraction.

Phi-4 direct changes from $6/6$ exact table goals to $1/6$ mapping goals. Both shared model mapping readouts score $0/6$, while shared rule readouts remain $6/6$. Saved extractions match across format controls, so these differences are not explained by different extracted facts in this cohort. No-history accuracy is $1/6$ for each model. The controls expose history and representation dependence, without establishing a broad model ranking.

6.3. Goal-level agent and program adapters

Phi-4 direct and facts + rule each score $6/6$; the ReAct adapter scores $5/6$, with one missing or invalid goal. The CaP adapter



(a) Shared undo-start object state (b) Book adopted: return the cup (c) Cup adopted: return the book

Fig. 3. Recorded Pomaria images and structured-rule undo branches. Storage is on the left and the activity support on the right. Only the adoption declaration differs between branches. The robot occludes some protected objects in (b); preservation is checked from traces, not images.

Table 4. History-generation outcomes for the selected geometry-based generator: 16 development candidates from 15 recipe groups. Every candidate remains in the denominator at every stage.

Stage	Successful / selected
Layout construction	16/16
Initialization preview	16/16
Complete physical history	10/16

uses twenty-two calls and returns three schema-valid goals, none correct. Two protect everything despite required restorations; one assigns restoration to protected objects. The other three conditions yield no valid goal. All six conditions remain in its denominator.

These results distinguish program execution from correct historical attribution: a valid returned contract can still encode the wrong undo boundary. They characterize the implemented goal-level adapters. The comparison does not reproduce original CaP's full prompting, libraries and robot API, and no adapter-specific physical batch is reported.

6.4. Development-time physical transfer

All sixteen reconstructed layouts pass construction and preview; ten produce valid histories with the selected controller. The six failures involve contact candidate selection, constrained extraction, loaded-arm retreat, release checks or base/grasp search. Terminal cleanup errors can obscure earlier primitive errors, motivating separate error logging.

A single geometry policy produces complete histories in ten of the sixteen conditions, providing evidence of reuse across the fixed cohort. Passing layout construction and initialization does not guarantee a complete manipulation history. Independent held-out reliability and paired undo for this cohort remain unmeasured.

7. LIMITATIONS AND CONCLUSION

The task currently assumes complete authored evidence, exact identities, known supports and persistent adoption declarations. Three physical ancestors, correlated branches and a single execution seed limit the main comparison. Models differ in size, and goal formatting materially affects their results. Sticky grasping, privileged geometry and sampled protection checks restrict claims about real-world contact manipulation. There are no independent human annotations, concurrent external actions, or tests of irreversible processes.

The development transfer cohort demonstrates reuse across several scene, asset and history-combination conditions. Because those cases informed debugging, independently selected cases are required to measure unseen performance after the present improvements. The next validation is complete history-plus-paired-undo regression, followed by a newly frozen cross-factor test and physical comparison of the goal adapters.

Task-Scoped Physical Undo connects historical responsibility to physical restoration and protection. Matched-state histories and separate G, E, T, J measurements reveal mistakes hidden by predicted-goal completion alone. The current facts–rule results establish an executable reference on six English conditions, while the ten-of-sixteen history-generation result exposes the remaining physical bottleneck. The contribution is a concrete task and auditable evaluation, with broader generalization still open.

Code repository. The implementation and runnable examples are maintained at `https://github.com/oneoutofseven/task-scoped-undo`.

8. REFERENCES

[1] Bin Shao, Du Li, and Ning Gu, "An algorithm for selective undo of any operation in collaborative applications,"

in *Proceedings of the ACM International Conference on Supporting Group Work*, 2010, pp. 131–140.

[2] James Harland, David N. Morley, John Thangarajah, and Neil Yorke-Smith, "Aborting, suspending, and resuming goals and plans in BDI agents," *Autonomous Agents and Multi-Agent Systems*, vol. 31, no. 2, pp. 288–331, 2017.

[3] Darius Has, Adrian Groza, and Mihai Pomarlan, "Handling abort commands for household kitchen robots," arXiv:2408.14480, 2024.

[4] Gehan Zheng, Sanjay Seenivasan, Matthew Johnson-Roberson, and Weiming Zhi, "Rewind-IL: Online failure detection and state respawning for imitation learning," arXiv:2604.16683, 2026.

[5] Chengshu Li, Ruohan Zhang, Josiah Wong, et al., "BEHAVIOR-1K: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation," in *Proceedings of the 6th Conference on Robot Learning*, 2023, vol. 205 of *Proceedings of Machine Learning Research*, pp. 80–93.

[6] Matthew Chang, Gunjan Chhablani, Alexander Clegg, et al., "PARTNR: A benchmark for planning and reasoning in embodied multi-agent tasks," arXiv:2411.00081, 2024.

[7] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[8] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng, "Code as policies: Language model programs for embodied control," arXiv:2209.07753, 2022.

[9] Caelan Reed Garrett, Tomás Lozano-Pérez, and Leslie Pack Kaelbling, "PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning," in *International Conference on Automated Planning and Scheduling*, 2020.

[10] MoveIt Project, "MoveIt Task Constructor documentation," `https://moveit.github.io/moveit_task_constructor/`, 2026, Accessed October 20, 2026.

[11] Aidan Curtis, Xiaolin Fang, Leslie Pack Kaelbling, Tomás Lozano-Pérez, and Caelan Reed Garrett, "Long-horizon manipulation of unknown objects via task and motion planning with estimated affordances," arXiv:2108.04145, 2021.

[12] Balakumar Sundaralingam, Siva Kumar Sastry Hari, Adam Fishman, et al., "cuRobo: Parallelized collision-free minimum-jerk robot motion generation," arXiv:2310.17274, 2023.

[13] Shuai Bai, Yuxuan Cai, Ruizhe Chen, et al., "Qwen3-VL technical report," arXiv:2511.21631, 2025.

[14] Marah Abdin, Jyoti Aneja, Harkirat Behl, et al., "Phi-4 technical report," arXiv:2412.08905, 2024.